코드 예제

개요

이 문서는 AMD SEV 라이브러리의 주요 기능별로 완전한 Rust 코드 예제를 제공합니다. 모든 예제는 실제로 동작하며, 단계별 구현 가이드와 함께 제공됩니다.

기본 설정

Cargo.toml 설정

```
[package]
name = "sev-examples"
version = "0.1.0"
edition = "2021"

[dependencies]
sev = { version = "1.2.1", features = ["openssl", "sev", "snp"] }
tokio = { version = "1.0", features = ["full"] }
anyhow = "1.0"
hex = "0.4"
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
```

```
[features]

dangerous_hw_tests = []
```

공통 임포트

```
use sev::{

error::UserApiError,

firmware::host::{Firmware, Status, State},

Generation,

};

use anyhow::Result;

use std::fs::OpenOptions;
```

1. 플랫폼 관리 예제

1.1 기본 플랫폼 상태 확인

```
use sev::firmware::host::{Firmware, Status, State};


fn main() -> Result<(), Box<dyn std::error::Error>> {

// SEV 펌웨어 핸들 생성

let mut firmware = Firmware::open()?;

// 플랫폼 상태 조회
```

```
let status: Status = firmware.platform_status()?;
println!("=== SEV 플랫폼 상태 ===");
println!("상태: {:?}", status.state);
println!("활성 게스트 수: {}", status.guests);
println!("펌웨어 버전: {}.{}.{}",
status.build.version.major,
status.build.version.minor,
status.build.build
);
// 상태에 따른 처리
match status.state {
State::Uninitialized => {
println!("플랫폼이 초기화되지 않았습니다.");
println!("platform_reset()을 호출하여 초기화하세요.");
},
State::Initialized => {
println!("플랫폼이 초기화되었습니다. 게스트 런치가 가능합니다.");
},
State::Working => {
println!("플랫폼이 작업 중입니다. {} 개의 게스트가 활성화되어 있습니다.",
status.guests);
},
}
```

```
Ok(())
}
```

1.2 완전한 플랫폼 초기화

```
use sev::{
firmware::host::{Firmware, Identifier},
certs::sev::sev::{Certificate, Chain},
error::UserApiError,
};

fn initialize_sev_platform() -> Result<(), UserApiError> {
let mut firmware = Firmware::open()
.map_err(|e| UserApiError::IOError(e))?;
println!("1. 플랫폼 상태 확인 중...");
let status = firmware.platform_status()?;
println!(" 현재 상태: {:?}", status.state);
// 필요한 경우 플랫폼 리셋
if status.guests > 0 {
println!("2. 활성 게스트가 있습니다. 플랫폼을 리셋합니다...");
firmware.platform_reset()?;
println!(" 플랫폼 리셋 완료");
}
```

```
println!("3. PEK (Platform Encryption Key) 생성 중...");
firmware.pek_generate()?;
println!(" PEK 생성 완료");
println!("4. PDH (Platform Diffie-Hellman) 키 쌍 생성 중...");
firmware.pdh_generate()?;
println!(" PDH 키 쌍 생성 완료");
println!("5. 인증서 체인 내보내기...");
let chain: Chain = firmware.pdh_cert_export()?;
println!(" 인증서 체인 내보내기 완료");
println!(" - PDH: 사용법 = {:?}", chain.pdh);
println!(" - PEK: 사용법 = {:?}", chain.pek);
println!(" - OCA: 사용법 = {:?}", chain.oca);
println!(" - CEK: 사용법 = {:?}", chain.cek);
println!("6. 플랫폼 식별자 조회...");
let identifier: Identifier = firmware.get_identifier()?;
println!(" 플랫폼 ID: {}", identifier);
println!("✅ SEV 플랫폼 초기화 완료!");
Ok(())
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
match initialize_sev_platform() {
Ok(()) => println!("플랫폼 초기화 성공"),
```

```
Err(e) => {
eprintln!("플랫폼 초기화 실패: {}", e);
return Err(e.into());
}
}
Ok(())
}
```

1.3 인증서 검증 예제

```
use sev::{
firmware::host::Firmware,
certs::sev::{sev::Chain, Verifiable},
Generation,
};
use std::convert::TryFrom;

fn verify_certificate_chain() -> Result<(), Box<dyn std::error::Error>> {
let mut firmware = Firmware::open()?;
// 인증서 체인 내보내기
println!("인증서 체인 내보내기 중...");
let chain: Chain = firmware.pdh_cert_export()?;
// 세대 식별
```

```
println!("프로세서 세대 식별 중...");
let  generation = Generation::try_from(&chain)
.map_err(|_| "프로세서 세대를 식별할 수 없습니다")?;
println!("식별된 세대: {}", generation.titlecase());
// 인증서 체인 검증
println!("인증서 체인 검증 중...");
match  chain.verify() {
Ok(()) => {
println!("✅ 인증서 체인 검증 성공!");
println!(" - 모든 인증서가 유효합니다");
println!(" - 서명 체인이 올바릅니다");
},
Err(e) => {
println!("❌ 인증서 체인 검증 실패: {}", e);
return Err(e.into());
}
}
// 개별 인증서 정보 출력
println!("\n=== 인증서 정보 ===");
println!("PDH (Platform Diffie-Hellman):");
println!(" - 용도: 키 교환");
println!(" - 상태: 유효");
println!("PEK (Platform Encryption Key):");
```

```
println!(" - 용도: 플랫폼 암호화");
println!(" - 상태: 유효");
println!("OCA (Owner Certificate Authority):");
println!(" - 용도: 소유자 인증");
println!(" - 상태: 유효");
println!("CEK (Chip Endorsement Key):");
println!(" - 용도: 칩 보증");
println!(" - 상태: 유효");
Ok(())
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
verify_certificate_chain()
}
```

2. SEV-SNP 플랫폼 관리 예제

2.1 SNP 플랫폼 상태 및 설정

```
use sev::{
firmware::host::{Firmware, SnpPlatformStatus, Config, TcbVersion, MaskId},
Generation,
};
```

```rust
fn manage_snp_platform() -> Result<(), Box<dyn std::error::Error>> {
let mut firmware = Firmware::open()?;
// 호스트 세대 식별
println!("1. 호스트 프로세서 세대 식별 중...");
let generation = Generation::identify_host_generation()?;
println!(" 식별된 세대: {}", generation.titlecase());
// SNP 플랫폼 상태 조회
println!("2. SNP 플랫폼 상태 조회 중...");
let status: SnpPlatformStatus = firmware.snp_platform_status()?;
println!("=== SNP 플랫폼 상태 ===");
println!("펌웨어 버전: {}.{}", status.version.0, status.version.1);
println!("빌드 ID: {}", status.build_id);
println!("활성 게스트 수: {}", status.guest_count);
println!("플랫폼 상태: {}", status.state);
println!("플랫폼 TCB 버전:");
println!(" - 마이크로코드: {}", status.platform_tcb_version.microcode);
println!(" - SNP: {}", status.platform_tcb_version.snp);
println!(" - TEE: {}", status.platform_tcb_version.tee);
println!(" - 부트로더: {}", status.platform_tcb_version.bootloader);
println!("보고된 TCB 버전:");
println!(" - 마이크로코드: {}", status.reported_tcb_version.microcode);
println!(" - SNP: {}", status.reported_tcb_version.snp);
```

```
println!(" - TEE: {}", status.reported_tcb_version.tee);

println!(" - 부트로더: {}", status.reported_tcb_version.bootloader);

// SNP 설정 업데이트 (주의: 실제 하드웨어에서만 테스트)

#[cfg(feature = "dangerous_hw_tests")]

{

println!("3. SNP 설정 업데이트 중...");

let  new_config = Config::new(

TcbVersion::new(

Some(status.reported_tcb_version.microcode),

status.reported_tcb_version.snp,

status.reported_tcb_version.tee,

status.reported_tcb_version.bootloader,

0, // 예약됨

),

MaskId(0), // 마스크 없음

);

firmware.snp_set_config(new_config)?;

println!(" SNP 설정 업데이트 완료");

println!("4. SNP 설정 커밋 중...");

firmware.snp_commit()?;

println!(" SNP 설정 커밋 완료");

}

println!("✅ SNP 플랫폼 관리 완료!");
```

```
Ok(())
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
manage_snp_platform()
}
```

2.2 VLEK Hashstick 관리

```
use sev::{
firmware::host::{Firmware, WrappedVlekHashstick},
Generation,
};
use std::fs;

fn load_vlek_hashstick() -> Result<(), Box<dyn std::error::Error>> {
let mut firmware = Firmware::open()?;
// 호스트 세대 식별
println!("1. 호스트 프로세서 세대 식별 중...");
let generation = Generation::identify_host_generation()?;
println!(" 식별된 세대: {}", generation.titlecase());
// VLEK Hashstick 파일 읽기 (예제용 - 실제로는 적절한 경로 사용)
println!("2. VLEK Hashstick 파일 읽기 중...");
```

```
let hashstick_path = "vlek_hashstick.bin"; // 실제 파일 경로로 변경
// 파일이 존재하는지 확인
if !std::path::Path::new(hashstick_path).exists() {
println!("⚠️ VLEK Hashstick 파일이 없습니다: {}", hashstick_path);
println!(" 실제 환경에서는 AMD에서 제공받은 VLEK Hashstick을 사용하세요.");
return Ok(());
}
let hashstick_bytes = fs::read(hashstick_path)?;
println!(" 파일 크기: {} 바이트", hashstick_bytes.len());
// Hashstick 파싱 및 검증
println!("3. VLEK Hashstick 파싱 및 검증 중...");
let hashstick = WrappedVlekHashstick::from_bytes(
hashstick_bytes.as_slice(),
generation,
)?;
println!(" VLEK Hashstick 파싱 완료");
// Hashstick 로드
println!("4. VLEK Hashstick 로드 중...");
firmware.snp_vlek_load(hashstick)?;
println!(" VLEK Hashstick 로드 완료");
println!("✅ VLEK Hashstick 관리 완료!");
Ok(())
}
```

```
fn main() -> Result<(), Box<dyn std::error::Error>> {
load_vlek_hashstick()
}
```

3. SEV 게스트 런치 예제

3.1 기본 SEV 게스트 런치

```
use sev::{
launch::sev::{Launcher, New, Start, Policy, PolicyFlags, Session, Measurement},
firmware::host::Version,
certs::sev::sev::Certificate,
};
use std::{fs::OpenOptions, mem::MaybeUninit};

fn launch_sev_guest() -> Result<(), Box<dyn std::error::Error>> {
// KVM 및 SEV 파일 디스크립터 열기
println!("1. KVM 및 SEV 디바이스 열기...");
let kvm_fd = OpenOptions::new()
.read(true)
.write(true)
.open("/dev/kvm")?;
```

```
let sev_fd = OpenOptions::new()
.read(true)
.write(true)
.open("/dev/sev")?;
println!(" 디바이스 열기 완료");
// SEV 런처 생성
println!("2. SEV 런처 초기화 중...");
let launcher: Launcher<New, _, _> = Launcher::new(kvm_fd, sev_fd)?;
println!(" 런처 초기화 완료");
// 게스트 정책 설정
println!("3. 게스트 정책 설정 중...");
let policy = Policy {
flags: PolicyFlags::NO_DEBUG | PolicyFlags::ENCRYPTED_STATE,
minfw: Version { major: 1, minor: 0 },
};
// 세션 정보 설정 (실제 구현에서는 적절한 키 교환 필요)
let session = Session {
nonce: [0u8; 16],
wrap_tk: [0u8; 32],
wrap_iv: [0u8; 16],
wrap_mac: [0u8; 32],
policy_mac: [0u8; 32],
};
```

```
// 테넌트 DH 인증서 (실제 구현에서는 적절한 인증서 사용)
let tenant_cert: Certificate = unsafe { MaybeUninit::zeroed().assume_init() };
let start = Start {
policy,
cert: tenant_cert,
session,
};
println!(" 정책 설정 완료");
println!(" - 디버그 비활성화: {}", policy.flags.contains(PolicyFlags::NO_DEBUG));
println!(" - 암호화된 상태: {}", policy.flags.contains(PolicyFlags::ENCRYPTED_STAT
// 게스트 시작
println!("4. 게스트 시작 중...");
let mut launcher = launcher.start(start)?;
println!(" 게스트 시작 완료");
// 게스트 메모리 데이터 암호화 (예제 데이터)
println!("5. 게스트 메모리 암호화 중...");
let guest_memory = vec![0u8; 4096]; // 4KB 페이지
launcher.update_data(&guest_memory)?;
println!(" 메모리 암호화 완료 ({} 바이트)", guest_memory.len());
// SEV-ES의 경우 VMSA 업데이트
println!("6. VMSA 업데이트 중...");
launcher.update_vmsa()?;
println!(" VMSA 업데이트 완료");
```

```
// 측정 요청
println!("7. 게스트 측정 중...");
let launcher = launcher.measure()?;
let measurement: Measurement = launcher.measurement();
println!(" 측정 완료");
println!(" - 측정값: {}", hex::encode(measurement.measure));
println!(" - 논스: {}", hex::encode(measurement.mnonce));
// 런치 완료
println!("8. 게스트 런치 완료 중...");
let handle = launcher.finish()?;
println!(" 게스트 런치 완료");
println!("✅ SEV 게스트 런치 성공!");
Ok(())
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
launch_sev_guest()
}
```

3.2 SEV-ES 게스트 런치

```
use sev::{
launch::sev::{Launcher, New, Start, Policy, PolicyFlags, Session, Secret, Header
```

```
firmware::host::Version,

certs::sev::sev::Certificate,

};

use std::{fs::OpenOptions, mem::MaybeUninit};



fn launch_sev_es_guest() -> Result<(), Box<dyn std::error::Error>> {

// 디바이스 열기

println!("1. 디바이스 초기화 중...");

let kvm_fd = OpenOptions::new()

.read(true)

.write(true)

.open("/dev/kvm")?;

let sev_fd = OpenOptions::new()

.read(true)

.write(true)

.open("/dev/sev")?;

// SEV-ES 런처 생성

println!("2. SEV-ES 런처 초기화 중...");

let launcher: Launcher<New, _, _> = Launcher::new_es(kvm_fd, sev_fd)?;

println!(" SEV-ES 런처 초기화 완료");

// 정책 설정 (SEV-ES는 ENCRYPTED_STATE 필수)

let policy = Policy {

flags: PolicyFlags::NO_DEBUG | PolicyFlags::ENCRYPTED_STATE | PolicyFlags::NO_SE
```

```
minfw: Version { major: 1, minor: 0 },
};
let session = Session {
nonce: [1u8; 16], // 실제로는 랜덤 논스 사용
wrap_tk: [0u8; 32],
wrap_iv: [0u8; 16],
wrap_mac: [0u8; 32],
policy_mac: [0u8; 32],
};
let tenant_cert: Certificate = unsafe { MaybeUninit::zeroed().assume_init() };
let start = Start {
policy,
cert: tenant_cert,
session,
};
println!("3. SEV-ES 게스트 시작 중...");
let mut launcher = launcher.start(start)?;
// 게스트 코드 및 데이터 암호화
println!("4. 게스트 코드 암호화 중...");
let guest_code = vec![0x90u8; 4096]; // NOP 명령어로 채운 4KB
launcher.update_data(&guest_code)?;
println!("5. 게스트 데이터 암호화 중...");
let guest_data = vec![0u8; 8192]; // 8KB 데이터
```

```
launcher.update_data(&guest_data)?;

// VMSA (Virtual Machine Save Area) 암호화

println!("6. VMSA 암호화 중...");

launcher.update_vmsa()?;

println!(" VMSA 암호화 완료 (레지스터 상태 보호됨)");

// 측정

println!("7. 게스트 측정 중...");

let launcher = launcher.measure()?;

let measurement = launcher.measurement();

println!(" SEV-ES 측정 완료");

println!(" - 코드 + 데이터 + VMSA 측정값: {}", hex::encode(measurement.measure));

// 시크릿 주입 (선택사항)

println!("8. 시크릿 주입 중...");

let secret_data = b"This is a secret for the guest";

let secret = Secret {

header: Header {

flags: HeaderFlags::default(),

iv: [2u8; 16],

mac: [0u8; 32],

},

ciphertext: secret_data.to_vec(),

};

let guest_secret_address = 0x1000; // 게스트 메모리 주소
```

```
launcher.inject(&secret, guest_secret_address)?;

println!(" 시크릿 주입 완료 (주소: 0x{:x})", guest_secret_address);

// 런치 완료

println!("9. SEV-ES 게스트 런치 완료 중...");

let  handle = launcher.finish()?;

println!("✅ SEV-ES 게스트 런치 성공!");

println!(" 게스트는 이제 실행 준비가 완료되었습니다.");

println!(" - 메모리 암호화: 활성");

println!(" - 레지스터 보호: 활성");

println!(" - 디버그 방지: 활성");

Ok(())

}



fn  main() -> Result<(), Box<dyn  std::error::Error>> {

launch_sev_es_guest()

}
```

4. SEV-SNP 게스트 런치 예제

4.1 기본 SNP 게스트 런치

```
use sev::{

launch::snp::{Launcher, New, Start, Update, PageType, Finish},
```

```
firmware::guest::GuestPolicy,
};
use std::fs::OpenOptions;


const KVM_SEV_SNP_FINISH_DATA_SIZE: usize = 32;


fn launch_snp_guest() -> Result<(), Box<dyn std::error::Error>> {
// 디바이스 초기화
println!("1. SEV-SNP 디바이스 초기화 중...");
let kvm_fd = OpenOptions::new()
.read(true)
.write(true)
.open("/dev/kvm")?;
let sev_fd = OpenOptions::new()
.read(true)
.write(true)
.open("/dev/sev")?;
// SNP 런처 생성
println!("2. SNP 런처 초기화 중...");
let launcher: Launcher<New, _, _> = Launcher::new(kvm_fd, sev_fd)?;
println!(" SNP 런처 초기화 완료");
// 게스트 정책 설정
println!("3. 게스트 정책 설정 중...");
```

```rust
let guest_policy = GuestPolicy::default();
let gosvw = [0u8; 16]; // Guest OS Visible Workarounds
let start = Start::new(guest_policy, gosvw);
println!(" 정책 설정 완료");
// 게스트 시작
println!("4. SNP 게스트 시작 중...");
let mut launcher = launcher.start(start)?;
println!(" SNP 게스트 시작 완료");
// 일반 페이지 업데이트
println!("5. 일반 페이지 업데이트 중...");
let normal_page_data = vec![0x90u8; 4096]; // NOP으로 채운 페이지
let update = Update::new(
0x1000, // 게스트 프레임 번호
&normal_page_data, // 페이지 데이터
PageType::Normal, // 일반 페이지
);
let gpa = 0x1000; // 게스트 물리 주소
let gpa_len = 4096; // 페이지 크기
launcher.update_data(update, gpa, gpa_len)?;
println!(" 일반 페이지 업데이트 완료");
// 제로 페이지 업데이트
println!("6. 제로 페이지 업데이트 중...");
let zero_page_data = vec![0u8; 4096];
```

```
let zero_update = Update::new(
0x2000,
&zero_page_data,
PageType::Zero,
);
launcher.update_data(zero_update, 0x2000, 4096)?;
println!(" 제로 페이지 업데이트 완료");
// VMSA 페이지 업데이트
println!("7. VMSA 페이지 업데이트 중...");
let vmsa_data = vec![0u8; 4096]; // 실제로는 적절한 VMSA 데이터 사용
let vmsa_update = Update::new(
0x3000,
&vmsa_data,
PageType::Vmsa,
);
launcher.update_data(vmsa_update, 0x3000, 4096)?;
println!(" VMSA 페이지 업데이트 완료");
// 시크릿 페이지 업데이트 (선택사항)
println!("8. 시크릿 페이지 업데이트 중...");
let secrets_data = vec![0u8; 4096];
let secrets_update = Update::new(
0x4000,
&secrets_data,
```

```
PageType::Secrets,
);
launcher.update_data(secrets_update, 0x4000, 4096)?;
println!(" 시크릿 페이지 업데이트 완료");
// CPUID 페이지 업데이트
println!("9. CPUID 페이지 업데이트 중...");
let  cpuid_data = vec![0u8; 4096]; // 실제로는 CPUID 테이블 ◆◆◆이터
let  cpuid_update = Update::new(
0x5000,
&cpuid_data,
PageType::Cpuid,
);
launcher.update_data(cpuid_update, 0x5000, 4096)?;
println!(" CPUID 페이지 업데이트 완료");
// 런치 완료
println!("10. SNP 게스트 런치 완료 중...");
let  host_data = [0u8; KVM_SEV_SNP_FINISH_DATA_SIZE];
let  finish = Finish::new(
None, // ID 블록 (선택사항)
None, // ID 인증 (선택사항)
host_data, // 호스트 데이터
);
let (kvm_fd, sev_fd) = launcher.finish(finish)?;
```

```
println!("✅ SEV-SNP 게스트 런치 성공!");
println!(" 게스트는 이제 실행 준비가 완료되었습니다.");
println!(" - 메모리 암호화: 활성");
println!(" - 무결성 보호: 활성");
println!(" - 원격 증명: 지원");
Ok(())
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
launch_snp_guest()
}
```

5. 세션 관리 예제

5.1 보안 세션 설정

```
use sev::{
session::Session,
launch::sev::Policy,
certs::sev::{Chain, ca, sev},
};
use std::convert::TryFrom;
```

```rust
fn create_secure_session() -> Result<(), Box<dyn std::error::Error>> {
println!("1. 보안 세션 생성 중...");
// 정책 설정
let policy = Policy::default();
// 세션 생성
let session = Session::try_from(policy)?;
println!(" 세션 생성 완료");
// 인증서 체인 로드 (실제 환경에서는 적절한 인증서 사용)
println!("2. 인증서 체인 로드 중...");
// 예제용 더미 체인 (실제로는 유효한 인증서 체인 사용)
let chain = Chain {
ca: ca::Chain {
ark: ca::Certificate::default(),
ask: ca::Certificate::default(),
},
sev: sev::Chain {
cek: sev::Certificate::default(),
oca: sev::Certificate::default(),
pek: sev::Certificate::default(),
pdh: sev::Certificate::default(),
},
};
// 세션 시작
```

```
println!("3. 보안 세션 시작 중...");
let _active_session = session.start(chain)?;
println!(" 보안 세션 시작 완료");
println!("✅ 보안 세션 설정 완료!");
println!(" - 키 교환: 완료");
println!(" - 암호화 채널: 활성");
println!(" - 무결성 보호: 활성");
Ok(())
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
create_secure_session()
}
```

6. 종합 예제: 완전한 SEV 워크플로우

6.1 전체 SEV 플랫폼 및 게스트 관리

```
use sev::{
firmware::host::{Firmware, Status, State},
launch::sev::{Launcher, New, Start, Policy, PolicyFlags, Session},
certs::sev::sev::{Certificate, Chain},
error::UserApiError,
```

```
Generation,
};
use std::{fs::OpenOptions, mem::MaybeUninit, convert::TryFrom};


struct SevManager {
firmware: Firmware,
}


impl SevManager {
fn  new() -> Result<Self, UserApiError> {
let  firmware = Firmware::open()
.map_err(|e| UserApiError::IOError(e))?;
Ok(Self { firmware })
}
fn  initialize_platform(&mut  self) -> Result<(), UserApiError> {
println!("=== SEV 플랫폼 초기화 ===");
// 상태 확인
let  status = self.firmware.platform_status()?;
println!("현재 플랫폼 상태: {:?}", status.state);
// 필요시 리셋
if  matches!(status.state, State::Working) && status.guests > 0 {
println!("활성 게스트 종료를 위해 플랫폼 리셋 중...");
self.firmware.platform_reset()?;
```

```
}
// 키 생성
println!("PEK 생성 중...");
self.firmware.pek_generate()?;
println!("PDH 생성 중...");
self.firmware.pdh_generate()?;
println!("✅ 플랫폼 초기화 완료");
Ok(())
}
fn get_platform_info(&mut self) -> Result<(Chain, String), UserApiError> {
// 인증서 체인 내보내기
let chain = self.firmware.pdh_cert_export()?;
// 플랫폼 식별자
let identifier = self.firmware.get_identifier()?;
let id_string = format!("{}", identifier);
// 세대 식별
let generation = Generation::try_from(&chain)
.map_err(|_| UserApiError::Unknown)?;
println!("=== 플랫폼 정보 ===");
println!("프로세서 세대: {}", generation.titlecase());
println!("플랫폼 ID: {}", id_string);
Ok((chain, id_string))
}
```

```rust
fn launch_guest(&mut self) -> Result<(), Box<dyn std::error::Error>> {
println!("=== SEV 게스트 런치 ===");
// KVM 디바이스 열기
let kvm_fd = OpenOptions::new()
.read(true)
.write(true)
.open("/dev/kvm")?;
let sev_fd = OpenOptions::new()
.read(true)
.write(true)
.open("/dev/sev")?;
// 런처 생성
let launcher: Launcher<New, _, _> = Launcher::new(kvm_fd, sev_fd)?;
// 정책 및 세션 설정
let policy = Policy {
flags: PolicyFlags::NO_DEBUG | PolicyFlags::ENCRYPTED_STATE,
minfw: sev::firmware::host::Version { major: 1, minor: 0 },
};
let session = Session {
nonce: [1u8; 16],
wrap_tk: [0u8; 32],
wrap_iv: [0u8; 16],
wrap_mac: [0u8; 32],
```

```
policy_mac: [0u8; 32],
};
let tenant_cert: Certificate = unsafe { MaybeUninit::zeroed().assume_init() };
let start = Start {
policy,
cert: tenant_cert,
session,
};
// 게스트 시작
println!("게스트 시작 중...");
let mut launcher = launcher.start(start)?;
// 메모리 암호화
println!("게스트 메모리 암호화 중...");
let guest_memory = vec![0x90u8; 65536]; // 64KB
launcher.update_data(&guest_memory)?;
// VMSA 업데이트
println!("VMSA 업데이트 중...");
launcher.update_vmsa()?;
// 측정
println!("게스트 측정 중...");
let launcher = launcher.measure()?;
let measurement = launcher.measurement();
println!("측정값: {}", hex::encode(measurement.measure));
```

```
// 런치 완료
println!("게스트 런치 완료 중...");
let _handle = launcher.finish()?;
println!("✅ 게스트 런치 완료");
Ok(())
}
}


fn main() -> Result<(), Box<dyn std::error::Error>> {
println!("🚀 SEV 종합 관리 시스템 시작");
// SEV 매니저 생성
let mut manager = SevManager::new()?;
// 플랫폼 초기화
manager.initialize_platform()?;
// 플랫폼 정보 조회
let (_chain, _id) = manager.get_platform_info()?;
// 게스트 런치
manager.launch_guest()?;
println!("🎉 모든 작업 완료!");
Ok(())
}
```

7. 실행 가이드

7.1 환경 설정

```
# 필요한 권한 설정
sudo chmod 666 /dev/sev
sudo chmod 666 /dev/kvm

# 프로젝트 빌드
cargo build --release

# 기본 예제 실행
cargo run --example platform_status

# 하드웨어 테스트 (주의: 실제 SEV 하드웨어 필요)
cargo run --features dangerous_hw_tests --example full_workflow
```

7.2 디버깅 팁

```
// 로깅 활성화
env_logger::init();

// 상세한 오류 정보
match result {
```

```
Err(e) => {
eprintln!("오류 발생: {}", e);
eprintln!("오류 체인:");
let mut source = e.source();
while let Some(err) = source {
eprintln!(" 원인: {}", err);
source = err.source();
}
}
Ok(val) => println!("성공: {:?}", val),
}
```

이 코드 예제들은 AMD SEV 라이브러리의 주요 기능을 실제로 사용하는 방법을 보여줍니다. 각 예제는 단계별로 구성되어 있으며, 실제 환경에서 동작하도록 설계되었습니다.